

Lab 3 Report

Team: beluga

1. KV Server Architecture

Our KV server is implemented as a replicated state machine on top of Raft. Each server runs both a Raft peer and a KV service. For writes, the client sends the request to the current leader, the KV layer serializes the operation, and the server proposes it to Raft. Raft assigns the operation a log index, replicates it to a majority, and commits it. Once the committed entry reaches the apply path, the KV state machine updates the in-memory key/value map and the waiting RPC handler returns the result.

To connect proposal and completion cleanly, the RPC handler uses a per-log-index pending structure. After `propose`, the handler stores a waiter for the returned index. The apply thread processes committed entries, records the outcome for that index, and signals the waiter. This gives a simple `propose` → `apply` → `respond` flow without busy polling.

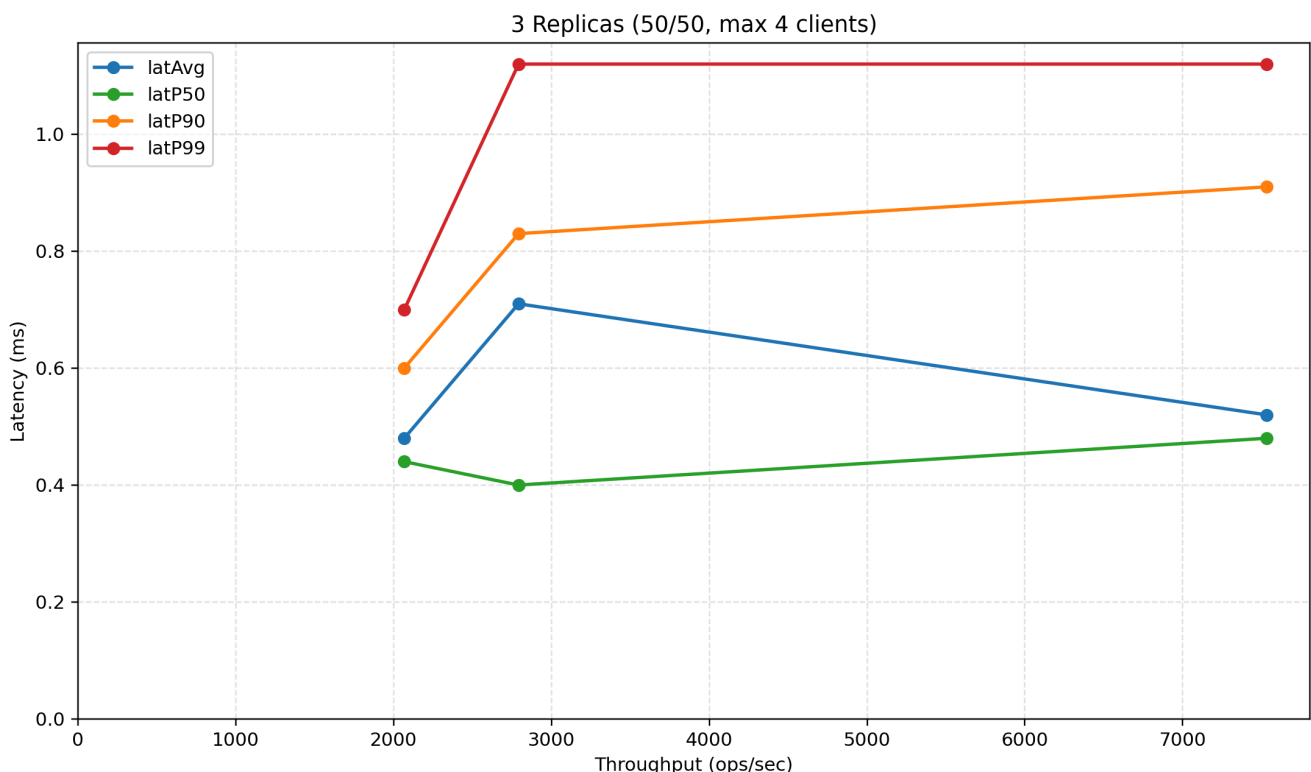
To make retries safe, the server uses the RIFL duplicate-detection pattern. Each client request includes a `client_id` and monotonically increasing `seq_num`. The server caches the result of each completed request keyed by this pair. If a client retries an operation, the server returns the cached result instead of executing it again, providing at-most-once semantics across failures and leader changes.

For reads, the implementation includes an optimized linearizable read path. When the leader can safely serve a read from local state, the server avoids sending `Get` through the full Raft write path. Otherwise, it falls back to the conservative Raft-backed path. This preserves correctness while improving read performance.

2. Latency and Throughput with 3 Replicas

For local evaluation, I generated throughput curves with up to 4 concurrent clients instead of 64 to keep benchmarking time manageable. The workload pattern is unchanged, but the reported throughput should be interpreted as the highest throughput observed in the local 4-client experiment rather than the system's absolute peak.

The figure below shows the latency-throughput curve for the 3-replica cluster.



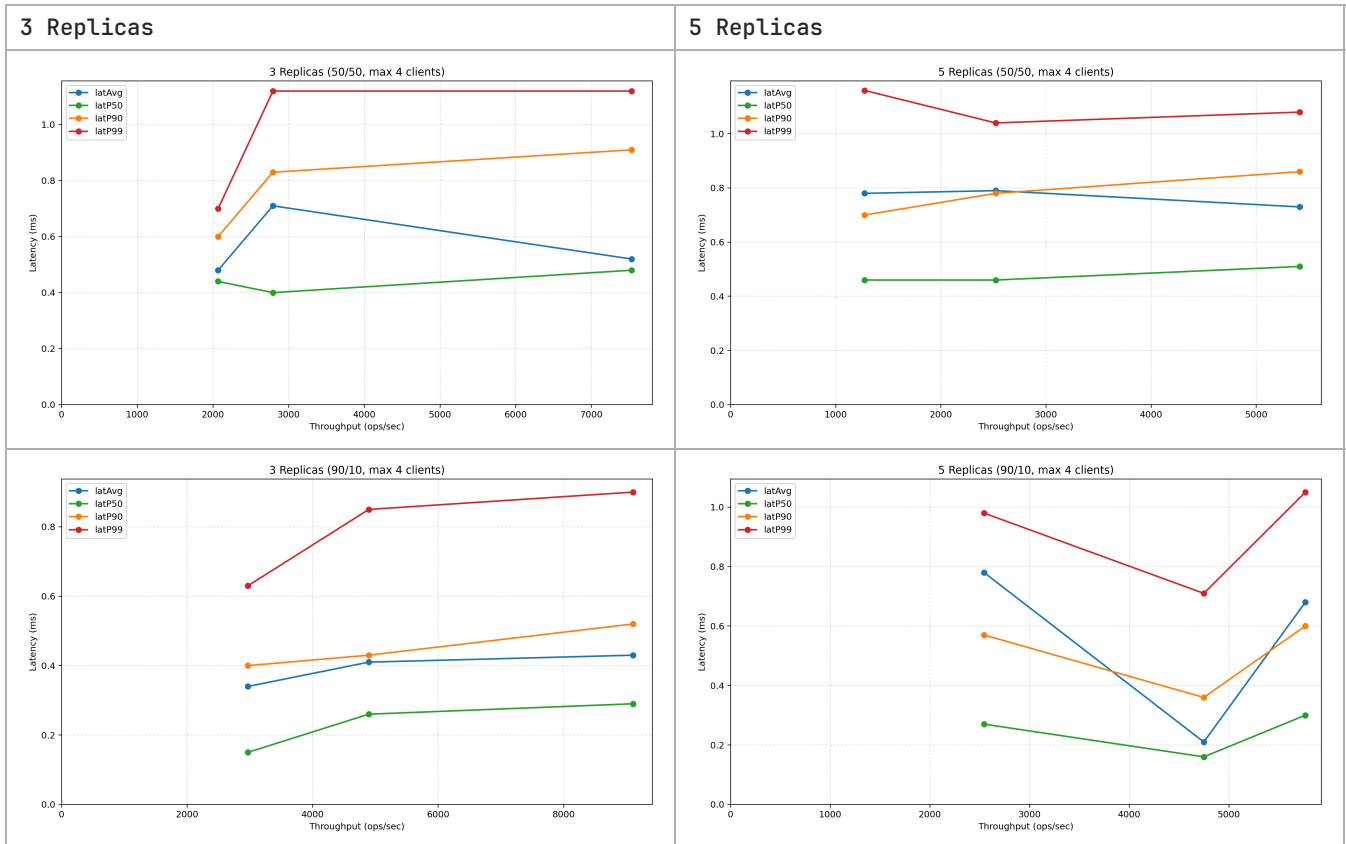
Key results from the final optimized 3-replica run are summarized below.

Metric	Measurement
Unloaded median Put latency	0.59 ms
Highest observed throughput at 50% Get / 50% Put (up to 4 clients)	7532.25 ops/sec
Highest observed throughput at 90% Get / 10% Put (up to 4 clients)	9105.00 ops/sec

At the 4-client operating point, the 3-replica cluster reached ops/sec on the 50/50 workload with ms p50 latency and ms p99 latency. On the 90/10 workload, it reached ops/sec with ms p50 latency and ms p99 latency. As expected, the read-heavier workload benefits more from the optimized read path.

3. Comparison of 3 vs. 5 Replicas

I repeated the throughput benchmark with both 3 replicas and 5 replicas, again using up to 4 clients for local runs. The plots below compare the two configurations.



The 5-replica configuration showed higher latency and lower throughput than the 3-replica configuration, especially for workloads containing writes. This is expected because each committed write needs a quorum, and the quorum size increases from 2 servers in a 3-node cluster to 3 servers in a 5-node cluster. The leader also has to manage more follower state, replication traffic, and heartbeat traffic.

The performance gap is largest for write-heavy and mixed workloads because these requests depend directly on Raft replication and commit completion. The gap is smaller for read-heavy workloads, especially after the read-path optimizations, but reads are still indirectly affected because the leader spends more resources coordinating with the larger cluster.

Representative local results are shown below.

Metric	3 Replicas	5 Replicas	Observation
Highest observed throughput (50/50, up to 4 clients)	7532.25	5412.20	5 replicas are about 28% lower.
Highest observed throughput (90/10, up to 4 clients)	9105.00	5766.51	5 replicas are about 37% lower.
Median unloaded Put latency	0.59 ms	0.62 ms	Single-client write latency increases only slightly.

In the 5-replica configuration, the 50/50 workload reached 5412.20 ops/sec at 4 clients with 0.51 ms p50 latency and 1.08 ms p99 latency. The 90/10 workload reached 5766.51 ops/sec at 4 clients with 0.30 ms p50 latency and 1.05 ms p99 latency. Compared with the 3-replica results, the dominant effect is reduced throughput rather than a dramatic increase in single-request latency.

4. Optimizations and Impact

The optimization effort focused on three goals: reducing KV-path overhead, avoiding unnecessary Raft work for linearizable reads, and improving leader-side replication efficiency. The final system includes the following main changes:

1. Replaced text-based KV command encoding with a compact binary representation.
2. Moved duplicate-request checks earlier so retries can return from cache without unnecessary Raft work.
3. Simplified duplicate-detection bookkeeping in the apply path.
4. Added a local linearizable read fast path for Get.
5. Added applied-index tracking so local reads are only served after the KV state machine reaches a safe point.
6. Added Raft-side read barrier support to obtain a safe read index without forcing every read through the full write path.
7. Added a conservative short read lease based on recent majority contact.
8. Fixed the leader wakeup path so newly proposed commands trigger replication immediately.
9. Stopped extra election work after a candidate already reached quorum.
10. Increased AppendEntries batch size for faster follower catch-up.
11. Replaced sequential leader replication with per-follower replication workers.
12. Switched replication signaling to an event-driven model.
13. Separated the ticker thread from the heavy replication path.
14. Reused recent majority-contact information for both read optimization and steady-state leader behavior.

These optimizations substantially improved performance. The table below compares the baseline implementation with the final optimized one using the same local benchmark setup.

Metric	Baseline	Final	Improvement
3-replica unloaded median Put latency	13.38 ms	0.59 ms	about 22.7x lower
3-replica best observed throughput, 50/50	449.61 ops/sec	7532.25 ops/sec	about 16.8x higher
3-replica best observed throughput, 90/10	444.18 ops/sec	9105.00 ops/sec	about 20.5x higher
5-replica best observed throughput, 50/50	587.38 ops/sec	5412.20 ops/sec	about 9.2x higher
5-replica best observed throughput, 90/10	551.51 ops/sec	5766.51 ops/sec	about 10.5x higher
5-replica single-client median Put latency proxy	8.54 ms	0.62 ms	about 13.8x lower

The baseline results show that the 50/50 and 90/10 workloads originally had similarly poor throughput, which is consistent with reads still paying almost the full cost of Raft coordination. After optimization, the 90/10 workload improved more than the 50/50 workload, matching the intended effect of the local linearizable read path and read-lease optimization.

The write-side improvements were also substantial. In the 3-replica cluster, unloaded median Put latency dropped from 13.38 ms to 0.59 ms after fixing the leader wakeup path and improving the replication pipeline. Throughput gains in both 3-replica and 5-replica experiments show that parallel follower replication and event-driven replication removed a major leader-side bottleneck.

For the 5-replica latency number, I used the single-client 100% Put run as a proxy because the dedicated latency.cpp benchmark is fixed to 3 replicas.